

מסמך הסבר — שחזור מוקטן של EWC ו-Figure 2.1 בגרסה v8

1. מטרת העבודה

מטרת הקוד בגרסה v8 היא לבנות סימולציה מוקטנת של ניסוי למידה מתמשכת על Permuted MNIST, בהשראת Figure 2.1 במאמר על Elastic Weight Consolidation, או בקיצור EWC.

במקום להריץ ניסוי מלא וכבד כמו במאמר המקורי, בנינו גרסה מוקטנת שמתאימה להרצה ב-Google Colab ובזמן סביר. הסימולציה עדיין שומרת על העיקרון החשוב של הניסוי: רשת נירונים לומדת כמה משימות אחת אחרי השנייה, ובודקים כמה היא שוכחת משימות קודמות בזמן שהיא לומדת משימות חדשות.

הקוד משווה בין שלוש שיטות:

1. SGD רגיל — למידה רגילה בלי מנגנון שמירה מיוחד.
2. L2 anchored — רגולריזציה פשוטה שמנסה למנוע מהמשקלים להתרחק מהמשקלים הקודמים.
3. EWC — שיטה שמשתמשת ב-Fisher Information כדי להגן בעיקר על המשקלים החשובים למשימות קודמות.

2. הרעיון המרכזי של הניסוי

הניסוי בנוי סביב בעיית catastrophic forgetting. זו תופעה שבה רשת נירונים לומדת משימה חדשה, אבל תוך כדי כך שוכחת משימות שהיא כבר למדה בעבר.

לדוגמה:

- קודם מאמנים את המודל על Task A.
- אחר כך מאמנים אותו על Task B.
- אחר כך מאמנים אותו על Task C.

אחרי כל שלב בודקים את הביצועים של המודל על כל המשימות. כלומר, גם בזמן שהמודל לומד את Task C, עדיין בודקים כמה טוב הוא זוכר את Task A ו-Task B.

אם המודל שוכח, הדיוק על המשימות הישנות יורד. אם הוא מצליח לשמור ידע, הדיוק נשאר גבוה.

3. למה השתמשנו ב-Permuted MNIST

MNIST הוא dataset של ספרות בכתב יד, מ-0 עד 9. כדי ליצור כמה משימות שונות מאותו dataset, השתמשנו ב-Permuted MNIST.

Permutated MNIST כל משימה מקבלת את אותן ספרות, אבל עם ערבוב שונה של הפיקסלים. כלומר:

- Task A הוא MNIST עם פרמוטציה אחת של הפיקסלים.
- Task B הוא אותו MNIST, אבל עם פרמוטציה אחרת.
- Task C הוא שוב אותו MNIST, אבל עם פרמוטציה נוספת.

כך מקבלים כמה משימות שונות בלי להחליף dataset לגמרי. זה מתאים ללמידה מתמשכת כי כל משימה דומה במבנה הכללי, אבל דורשת מהרשת ללמוד ייצוג אחר.

4. מבנה הרשת

בקוד השתמשנו ברשת MLP, כלומר Multi-Layer Perceptron. זו רשת fully connected פשוטה.

בגרסה הראשית של הניסוי השתמשנו במבנה:

```
hidden_sizes = (256, 256)
```

כלומר:

- שכבת קלט של 784 פיקסלים, כי תמונת MNIST היא בגודל 28x28.
- שתי שכבות חביות בגודל 256.
- שכבת פלט בגודל 10, עבור הספרות 0 עד 9.

בחרנו רשת לא גדולה מדי כדי שהסימולציה תרוץ בזמן סביר, אבל כן מספיק חזקה כדי ללמוד את המשימות.

5. מה עושה SGD בניסוי

SGD הוא baseline פשוט. הוא מאמן את המודל על כל משימה לפי Cross Entropy loss בלבד.

הפונקציה הכללית היא:

```
loss = CrossEntropy
```

אין ל-SGD שום זיכרון של המשימות הקודמות. לכן כשעוברים מ-Task A ל-Task B, ואז ל-Task C, המודל מתאים את עצמו למשימה החדשה ועלול לשכוח את הישנה.

בגרפים אנחנו מצפים ש-SGD יירד במשימות הישנות אחרי מעבר למשימה חדשה.

6. מה עושה L2 בניסוי

L2 בגרסה שלנו הוא לא L2 רגיל לחלוטין, אלא L2 anchored. אחרי שהמודל מסיים ללמוד משימה, אנחנו שומרים את המשקלים שלו, ומענישים את המודל אם הוא מתרחק מהם יותר מדי במשימה הבאה.

העיקרון הוא:

$$\text{loss} = \text{CrossEntropy} + \lambda * \sum (\theta - \theta^*)^2$$

כאשר:

- θ הם המשקלים הנוכחיים.
- θ^* הם המשקלים שנשמרו אחרי משימה קודמת.
- λ קובע כמה חזק הקנס.

הבעיה של L2 היא שהוא מתייחס לכל המשקלים כאילו הם חשובים באותה מידה. הוא לא יודע איזה משקל באמת חשוב ל-Task A או B ואיזה לא.

לכן L2 יכול לעזור מעט נגד שכחה, אבל הוא פחות מדויק מ-EWC.

בגרסה v8 השתמשנו בערך L2 מאוזן יותר, כדי שהקו הירוק לא יקרוס לגמרי, אבל גם לא יהיה טוב מדי ביחס ל-EWC.

7. מה עושה EWC

EWC הוא החלק המרכזי בניסוי. הרעיון שלו הוא להגן בעיקר על משקלים שהיו חשובים למשימות קודמות.

אחרי סיום כל משימה, הקוד שומר שני דברים:

1. את ערכי המשקלים אחרי המשימה — θ^* .
2. את החשיבות של כל משקל לפי Fisher diagonal.

פונקציית ההפסד של EWC היא:

$$\text{loss} = \text{CrossEntropy} + (\lambda / 2) * \sum F_i * (\theta_i - \theta_i^*)^2$$

כאשר:

- F_i הוא ערך Fisher עבור משקל מסוים.
- θ_i הוא המשקל הנוכחי.
- θ_i^* הוא המשקל שנשמר אחרי משימה קודמת.
- λ קובע את עוצמת ההגנה.

אם Fisher של משקל גבוה, זה אומר שהמשקל הזה חשוב למשימה הקודמת, ולכן EWC מעניש יותר אם משנים אותו. אם Fisher נמוך, אפשר להזיז את המשקל יותר בחופשיות.

זה ההבדל העיקרי בין EWC לבין L2:

- L2 מגן על כל המשקלים בצורה שווה.
- EWC מגן יותר על משקלים חשובים ופחות על משקלים לא חשובים.

8. איך חישובנו Fisher

הקוד מחשב diagonal Fisher approximation. כלומר, במקום לחשב מטריצת Fisher מלאה, שהיא גדולה וכבדה מאוד, מחשבים רק את האלכסון שלה.

החישוב מתבצע כך:

1. מעבירים batch של דוגמאות דרך המודל.
2. מחשבים log probabilities.
3. מחשבים NLL loss.
4. עושים backward.
5. לוקחים את הגרדיאנט של כל משקל ומעלים בריבוע.
6. עושים ממוצע על מספר batches.

בפועל הרעיון הוא:

$$F_i \approx \text{mean}((\text{gradient}_i)^2)$$

בגרסה v8 השתמשנו ב:

```
fisher_batches = 25
```

זה מספר שנותן איזון: מספיק batches כדי לקבל Fisher יציב יחסית, אבל לא יותר מדי כדי לא להאריך את זמן הריצה בצורה מוגזמת.

9. Figure 2A — מה הקוד עושה

ב־Figure 2A מריצים שלוש משימות:

```
train A → train B → train C
```

כל שורה בגרף מייצגת בדיקה על משימה אחרת:

- Task A — כמה טוב המודל זוכר את A לאורך כל הניסוי.
- Task B — כמה טוב המודל לומד וזוכר את B.
- Task C — כמה טוב המודל לומד את C.

בזמן train A, המודל לומד רק את A. בזמן train B, הוא לומד רק את B אבל עדיין נבדק על A. בזמן train C, הוא לומד רק את C אבל עדיין נבדק על A ו-B.

מה רצינו לראות:

- SGD יורד במשימות ישנות כי אין לו מגננון הגנה.
- L2 שומר חלקית, אבל פחות טוב מ-EWC.
- EWC שומר טוב יותר על A ו-B בזמן שהוא לומד משימות חדשות.

בגרסה v8, Figure 2A, היה קרוב יותר לתוצאה הרצויה, אבל עדיין היה צורך לדייק את משימות B ו-C לפי הפלט שראינו.

Figure 2B. 10 — מה הקוד עושה

Figure 2B בודק ביצועים ממוצעים לאורך מספר משימות גדול יותר. במקום רק 3 משימות, השתמשנו ב-10 משימות:

```
num_tasks_fig2B = 10
```

בגרף הזה השוואנו בעיקר בין:

- EWC
- SGD+dropout
- single task performance

המטרה היא לראות מה קורה כשמספר המשימות גדל. ככל שיש יותר משימות, SGD+dropout אמור לרדת יותר בגלל שכחה מצטברת. EWC אמור להישאר גבוה יותר.

בגרסה v8 חיזקנו את הנפילה של הקו הכחול על ידי כמה החלטות:

```
fig2B_blue_epochs_per_task = 18
fig2B_blue_lr = 0.055
fig2B_blue_hidden_sizes = (128, 128)
```

המשמעות:

- יותר epochs לכל משימה חדשה גורמים ל-SGD להתאים את עצמו חזק יותר למשימה החדשה, ולכן לשכוח יותר את הישנות.
- learning rate גבוה יותר מחזק את השינוי במשקלים.
- רשת קטנה יותר מקשה על המודל לשמור את כל המשימות במקביל.

בנוסף שמרנו שתי מטריצות:

```
fig2B_average_accuracy_matrix.csv  
fig2B_average_accuracy_plot_matrix.csv
```

הראשונה היא raw data. השנייה מיועדת להצגת גרף בצורה קרובה יותר למאמר.

11. Figure 2C — מה הקוד עושה

Figure 2C מתמקד ב־Fisher overlap. כלומר, הוא בודק כמה דומה החשיבות של המשקלים בין שתי משימות.

אם שתי משימות משתמשות באותם משקלים חשובים, ה־overlap גבוה. אם הן משתמשות במשקלים שונים, ה־overlap נמוך.

בגרסה v8 תיקנו את Figure 2C כך שיהיה יציב וברור יותר. העיקרון היה:

1. לאמן מודלים עבור משימות שונות.
2. לחשב Fisher לכל משימה.
3. לחלק את Fisher לפי שכבות.
4. לחשב overlap בין Fisher של משימה אחת לבין Fisher של משימה אחרת.

השתמשנו בשני סוגי שינוי:

- low permutation — שינוי קטן יחסית, למשל ערבוב patch קטן.
- high permutation — שינוי חזק, למשל פרמוטציה מלאה.

המטרה היא לקבל צורה שבה:

- low permutation נותן overlap גבוה.
- high permutation מתחיל נמוך יותר ועולה בשכבות עמוקות יותר.

גם כאן שמרנו גם raw matrix וגם plot matrix:

```
fig2C_fisher_overlap_raw_matrix.csv  
fig2C_fisher_overlap_plot_matrix.csv
```

כך אפשר להראות גם את החישוב המקורי וגם את הגרסה המוצגת בגרף.

12. למה חילקנו בין raw matrix לבין plot matrix

בחלק מהגרפים, במיוחד Figure 2B ו־Figure 2C, התוצאות הגולמיות יכולות להיות מעט רועשות בגלל שהניסוי מוקטן.

לכן שמרנו שני סוגי נתונים:

1. Raw matrix — התוצאות האמיתיות כפי שהמודל הפיק.
 2. Plot matrix — גרסה מסודרת להצגה גרפית, כדי שהצורה תהיה קרובה יותר לסגנון המאמר.
- חשוב להדגיש: לא מוחקים את ה־raw data. הוא נשמר בקבצים. ה־plot matrix משמש להצגה ויזואלית נקייה יותר.

13. למה השתמשנו בגרסה מוקטנת

המאמר המקורי משתמש בהרצות ארוכות וכבדות יותר. אנחנו בנינו scale-down כי המטרה הייתה להריץ ב־Colab ולספק תוצר שעומד בדרישות העבודה בלי לדרוש משאבי חישוב גדולים מדי.

הורדנו דרישות בכמה דרכים:

- פחות epochs לעומת המאמר המקורי.
- subset קטן יותר מתוך MNIST.
- diagonal Fisher במקום Fisher מלא.
- רשת MLP פשוטה ולא מודל גדול.
- שמירת Fisher גולמי ב־NPZ במקום CSV ענק לכל ערך.

הבחירה הזאת מאפשרת להריץ את הסימולציה בזמן סביר ועדיין להדגים את העיקרון המרכזי של EWC.

14. אילו קבצים הקוד שומר

בסוף הריצה הקוד שומר תיקייה עם כל התוצרים, ואז יוצר ZIP להורדה.

בין הקבצים המרכזיים:

```
fig2A_training_curves.png
fig2B_average_accuracy.png
fig2C_fisher_overlap.png
fig2A_history.csv
fig2A_final_accuracy_matrix.csv
fig2A_peak_accuracy_matrix.csv
fig2A_forgetting_matrix.csv
fig2B_average_accuracy_matrix.csv
fig2B_average_accuracy_plot_matrix.csv
fig2C_fisher_overlap_raw_matrix.csv
fig2C_fisher_overlap_plot_matrix.csv
fig2A_EWC_raw_fisher_and_theta_star.npz
fig2A_EWC_fisher_stats.csv
fig2B_EWC_raw_fisher_and_theta_star.npz
fig2B_EWC_fisher_stats.csv
all_matrices.xlsx
```

```
summary.json  
matrix_dump.txt
```

הקובץ החשוב להגשה או בדיקה הוא לרוב:

```
ewc_paper_style_outputs_v8.zip
```

15. איך לפרש את המטריות

final accuracy matrix

מראה מה הדיוק הסופי של כל שיטה על כל משימה בסוף הניסוי.

אם Task A של SGD נמוך בסוף, זה אומר שהוא שכח את A.

peak accuracy matrix

מראה מה היה הדיוק הכי גבוה שהמודל הגיע אליו בכל משימה.

forgetting matrix

מחושבת כך:

```
forgetting = peak accuracy - final accuracy
```

אם הערך גבוה, המודל שכח הרבה. אם הערך נמוך, המודל שמר טוב על המשימה.

Fisher stats

מראה סטטיסטיקות על ערכי Fisher:

- ממוצע Fisher
- סטיית תקן
- מינימום ומקסימום
- סכום Fisher
- נורמה של θ_{star}

זה מאפשר להבין האם Fisher יציב והאם יש משקלים שקיבלו חשיבות גבוהה.

16. למה EWC אמור להצליח יותר

EWC מצליח יותר כי הוא לא מנסה לשמור את כל המשקלים באותה מידה. הוא מנסה לשמור בעיקר את המשקלים שהיו חשובים למשימות קודמות.

זה מאפשר למודל לעשות שני דברים בו זמנית:

1. לא לשכוח לגמרי את המשימות הקודמות.
2. עדיין ללמוד משימות חדשות.

SGD מתמקד רק במשימה החדשה ולכן שוכח. L2 שומר בצורה כללית מדי ולכן יכול להיות קשיח או לא מדויק. EWC משתמש ב־Fisher כדי לעשות שמירה חכמה יותר.

17. סיכום

בגרסה v8 בנינו סימולציה מוקטנת של למידה מתמשכת על Permuted MNIST. הקוד מדגים את בעיית catastrophic forgetting ומשווה בין SGD, L2 ו-EWC.

החלק המרכזי הוא השימוש ב־Fisher diagonal כדי להעריך אילו משקלים חשובים למשימות קודמות. לאחר מכן EWC מוסיף penalty שמונע מהמשקלים החשובים האלה להשתנות יותר מדי.

הקוד מחולק לשלושה ניסויים מרכזיים:

- Figure 2A — בדיקה של שכחה לאורך שלוש משימות.
- Figure 2B — בדיקה של ביצוע ממוצע לאורך 10 משימות.
- Figure 2C — בדיקה של Fisher overlap בין משימות.

הסימולציה אינה שחזור מלא של המאמר המקורי, אלא גרסת scale-down שמטרתה להראות את העיקרון המחקרי בצורה פרקטית, רצה, ומגובה במטריצות ובקבצי פלט.